فعالیت کلاسی فصل ۳ — صفحه ۴۳ جزوه

طراحی معماری نرم‌افزار و قرارداد اولیه REST API

Project: Smart Team Task Manager (API Contract & Architecture)

ارائه‌دهنده	موضوع فعالیت	خروجی مورد انتظار
شروین آقائی	معماری سیستم و REST API	اولیه + دیاگرام API Contract

گام اول: شناسایی Resourceهای اصلی پروژه

استخراج موجودیت‌های کلیدی سیستم مدیریت وظایف تیمی بر اساس رویکرد Resource-Oriented

شروین آقائی

موجودیت‌ها و Resourceهای سیستم

بر اساس معماری **Modular Monolith** معرفی‌شده در فصل سوم (ماژول‌های Auth، Users، Tasks، Reports و Notifications)، منابع (Resources) پروژه عبارت‌اند از:

- **Users (`/api/users`):** مدیریت پروفایل کاربران، نقش‌های دسترسی (Admin, Manager, Member) و حساب‌های کاربری.
- **Tasks (`/api/tasks`):** هسته اصلی سیستم شامل ایجاد، تخصیص (Assign)، تغییر وضعیت (Todo, In-Progress, Done)، اولویت و حذف تسک‌ها.
- **Auth (`/api/auth`):** فرآیندهای ورود (Login)، ثبت‌نام و صدور توکن احراز هویت (JWT).
- **Reports (`/api/reports`):** گزارش‌گیری از عملکرد تیم و آمار تکمیل وظایف.
- **Notifications (`/api/notifications`):** ارسال اعلان خودکار هنگام تخصیص یا تغییر وضعیت تسک.

ساختار مدل داده (JSON Data Schema)

قالب استاندارد تبادل داده میان Client و Back-End:

```
// User Resource Model
{
  "id": "usr_101",
  "fullName": "Shervin Aghaei",
  "email": "shervin@team.ir",
  "role": "Manager"
}

// Task Resource Model
{
  "id": "tsk_504",
  "title": "طراحی قرارداد REST API",
  "description": "تعریف Endpointهای Users و Tasks",
  "status": "in_progress",
  "priority": "high",
  "assigneeId": "usr_101"
}
```

گام دوم و سوم: طراحی Endpointها، Method و Status Code

قرارداد اولیه API Contract برای دو منبع اصلی Users و Tasks (بدون افعال اضافی در URL)

شروین آقائی

۱. قرارداد Endpointهای منبع کاربران (Users Resource)

Method	Endpoint URL	شرح عملیات (Operation)	کدهای وضعیت (HTTP Status Codes)
GET	/api/users	دریافت لیست تمام کاربران ثبت‌نام‌شده در تیم	200 OK | 401 Unauthorized | 403 Forbidden
GET	/api/users/:id	دریافت اطلاعات جزئی یک کاربر خاص با شناسه id	200 OK | 404 Not Found | 401 Unauthorized
POST	/api/users	ایجاد و ثبت‌نام کاربر جدید در سیستم	201 Created | 400 Bad Request | 409 Conflict
PUT	/api/users/:id	به‌روزرسانی اطلاعات پروفایل یا نقش کاربر	200 OK | 400 Bad Request | 404 Not Found
DELETE	/api/users/:id	حذف حساب کاربری توسط مدیر سیستم	204 No Content | 403 Forbidden | 404 Not Found

۲. قرارداد Endpointهای منبع وظایف (Tasks Resource)

Method	Endpoint URL	شرح عملیات (Operation)	کدهای وضعیت (HTTP Status Codes)
GET	/api/tasks	دریافت لیست کل تسک‌ها (با قابلیت فیلتر وضعیت/اولویت)	200 OK | 401 Unauthorized | 500 Server Error

گام چهارم: دیاگرام معماری سیستم (Modular Monolith + REST API)

نمایش جریان درخواست (Request Flow) و مرزبندی منطقی ماژول‌ها در پروژه Smart Team Task Manager

شروین آقائی

جریان ارتباطی سرتاسری (End-to-End Request Flow)



ساختار داخلی هسته Back-End (معماری Modular Monolith — یک واحد استقرار)

تمام ماژول‌ها در یک برنامه واحد مستقر می‌شوند اما هر ماژول مسئولیت و منطق مستقل خود را دارد:

Auth Module
احراز هویت (JWT)
و کنترل مجوزها

Users Module
مدیریت حساب‌ها
و نقش‌های تیمی

Tasks Module
CRUD وظایف،
وضعیت و اولویت

Reports Module
گزارش پیشرفت
و آمار عملکرد

Notifications
ارسال هشدارهای
تغییر وضعیت

جمع‌بندی خروجی فعالیت فصل ۳ (API Contract اولیه)

دلایل فنی انتخاب معماری و اصول رعایت‌شده در طراحی رابط برنامه‌نویسی

شروین آقائی

چرا معماری Modular Monolith انتخاب شد؟

- **کاهش پیچیدگی عملیاتی:** در مقایسه با Microservices، نیازی به مدیریت چندین دیتابیس و ارتباطات شبکه‌ای پیچیده در فاز اول نیست.
- **مرزبندی شفاف کدها:** جداسازی ماژول‌های *Users* و *Tasks* باعث می‌شود تیم توسعه بدون تداخل روی بخش‌های مختلف کار کند.
- **آمادگی برای توسعه آینده:** در صورت رشد پروژه، هر ماژول به سادگی قابل تبدیل به یک Microservice مستقل است.

استانداردهای رعایت‌شده در API Contract

- **طراحی مبتنی بر منبع (Resource-Based):** استفاده از نام‌های جمع (`/users` و `/tasks`) به جای افعال در نشانی اینترنتی.
- **استفاده صحیح از HTTP Methods:** تفکیک دقیق عملیات خواندن (`GET`)، ایجاد (`POST`)، ویرایش (`PUT`) و حذف (`DELETE`).
- **پاسخ‌دهی استاندارد:** بازگرداندن کدهای وضعیت دقیق (`201` برای ایجاد موفق، `400` برای خطای ورودی و `404` برای عدم وجود منبع).